

Git & GitHub Guide

🔑 Git & GitHub — Beginner Guide

A practical, example-driven guide to Git and GitHub for DevOps engineers — from core concepts to real-world workflows and CI/CD basics.

📖 Table of Contents

1. [Git vs GitHub](#)
 2. [Repository](#)
 3. [Working Directory](#)
 4. [Staging Area](#)
 5. [Commit](#)
 6. [The Most Important Git Workflow](#)
 7. [Branch](#)
 8. [Merge](#)
 9. [Remote Repository](#)
 10. [HEAD](#)
 11. [.gitignore](#)
 12. [Git Commands](#)
 - [git init](#) · [git clone](#) · [git status](#) · [git add](#) · [git commit](#) · [git push](#) · [git pull](#) · [git fetch](#) · [git branch](#) · [git checkout](#) · [git switch](#) · [git merge](#) · [git log](#) · [git diff](#) · [git stash](#) · [git reset](#) · [git revert](#)
 13. [GitHub](#)
 - [Create a Repository](#) · [SSH Authentication](#) · [Personal Access Token](#) · [Pull Request](#) · [Branch Protection](#) · [README](#) · [GitHub Actions](#)
 14. [Complete Workflow Examples](#)
 15. [Git Architecture to Memorize](#)
 16. [Commands Cheat Sheet](#)
-

Git is one of the most important tools in DevOps — engineers constantly work with source code, configuration files, infrastructure code, CI/CD pipelines, and deployment files.

Think of Git as a **version-control system** that tracks every important change you make to your project.

1. Git vs GitHub

This is the first thing you should understand clearly.

Git

Git = Version Control System. Software installed on your computer that tracks changes in your files.

Example — index.html changes from Hello World to Hello DevOps. Git remembers: - Version 1 → Hello World - Version 2 → Hello DevOps

So if you make a mistake, you can go back to an older version.

GitHub

GitHub = Cloud platform for hosting Git repositories. Git works locally; GitHub stores your repository online and enables collaboration.

Git → Your computer → Tracks changes

GitHub → Internet/Cloud → Stores your Git repository → Collaboration

Simple analogy: Git is like Microsoft Word's version history. GitHub is like Google Drive, where you store and share the project.

Git ≠ GitHub — Git is the tool. GitHub is a platform that works with Git repositories.

2. Repository

A **repository (repo)** is a project that Git is tracking.

```
my-website/
├─ index.html
├─ style.css
├─ script.js
└─ README.md
```

When Git tracks this project, it becomes a **Git repository**, containing a hidden directory: `.git/` — Git's internal history data.

⚠ Don't manually modify `.git` unless you know exactly what you're doing.

3. Working Directory

The **working directory** is the actual files you're currently working on.

```
my-project/
├─ index.html
├─ style.css
└─ README.md
```

When you open and modify `index.html`, that change first exists in your working directory.

4. Staging Area

The **staging area** is where you tell Git: *"I want these changes to be included in my next commit."*

Example — you modify `index.html`, `style.css`, and `README.md`, but only want to commit the first two:

```
git add index.html
git add style.css
```

Working Directory → `git add` → Staging Area

5. Commit

A **commit** is a saved checkpoint in Git history.

```
git commit -m "Add homepage"
```

Think of a commit as: 📸 a snapshot of your project at a particular point in time.

Good commit messages: - Add login page - Fix database connection - Update Docker configuration

6. The Most Important Git Workflow

Memorize this:

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Local Repository
  ↓
git push
  ↓
GitHub Repository
```

This is extremely important for DevOps.

7. Branch

A **branch** allows you to work on changes separately from the main code.

```
main
├── commit 1
├── commit 2
└── commit 3
    └── feature-login
        ├── commit 4
        └── commit 5
```

You can develop the login feature without directly changing main.

Common branches: main, develop, feature-login, feature-payment, bugfix-login

8. Merge

After finishing your feature, you combine it with main — this is a **merge**.

```
main:      A — B —————┐
feature-:  C — D           │
                        ↓
                        merge
                        ↓
                        main
```

```
git switch main
git merge feature-login
```

Now the changes from feature-login are incorporated into main.

9. Remote Repository

A **remote repository** is a Git repository located somewhere else — usually GitHub.

Your Mac → Local Repository → git push → GitHub (Remote Repository)

Usually the remote is named origin. Check it with:

```
git remote -v
```

Example output:

```
origin  git@github.com:username/project.git
```

10. HEAD

HEAD = the commit/branch you are currently pointing to.

```
A ← B ← C
      ↑
    HEAD
```

If you're on main at commit C: HEAD → main → C. A new commit shifts HEAD forward:

```
A ← B ← C ← D
      ↑
    HEAD
```

Understanding HEAD matters most when learning `git reset`, `git checkout`, `git switch`, and `git revert`.

11. .gitignore

.gitignore tells Git: *“Don’t track these files.”*

```
node_modules/
.env
*.log
.DS_Store
```

Why? You usually don’t want to upload passwords, API keys, secret files, temporary files, large generated files, or dependencies.

Example — .env might contain:

```
DB_PASSWORD=secret123
AWS_ACCESS_KEY=xxxxx
```

⚠ You should **never** commit secrets to GitHub.

Git Commands

12. git init

Creates a new Git repository.

```
mkdir my-project
cd my-project
git init
```

Git creates my-project/.git/ and starts tracking the project. Check with:

```
git status
```

13. git clone

Copies an existing remote repository to your computer.

```
git clone https://github.com/username/project.git
```

Use clone when a project already exists on GitHub.

14. git status

Shows the current state of your repository.

```
git status
```

Example:

```
modified: index.html
untracked: app.js
```

Shows modified files, untracked files, staged files, and current branch. Use this command **very frequently**.

15. git add

Moves changes into the staging area.

```
git add index.html           # One file
git add index.html style.css # Multiple files
git add .                    # Everything
```

Working Directory → git add . → Staging Area

16. git commit

Creates a commit from staged changes.

```
git commit -m "Add homepage"
```

The -m means message.

Good	Bad
git commit -m "Add EC2 deployment script"	git commit -m "changes"

Your commit messages should describe what changed.

17. git push

Uploads your local commits to the remote repository.

```
git push
```

Example flow:

```
git add .
git commit -m "Add networking notes"
git push
```

18. git pull

Downloads changes from the remote repository and integrates them into your current branch.

```
git pull
```

Example: Your friend changes README.md and pushes it to GitHub. You run git pull to receive the changes.

19. git fetch

Downloads information about changes from the remote repository **without merging** them into your current branch.

```
git fetch
```

You can then inspect and decide what to do with the changes.

pull vs fetch

Command	Behavior
git fetch	Downloads remote changes; does not merge
git pull	Fetch + integrate changes

Simplified: git pull ≈ git fetch + integration

20. git branch

View branches:

```
git branch
```

```
* main
  feature-login
  development
```

The * marks your current branch. Create a branch:

```
git branch feature-login
```

21. git checkout

Historically used to switch branches:

```
git checkout feature-login
```

Create and switch in one step:

```
git checkout -b feature-login
```

22. git switch

Modern Git command specifically for branch switching.

```
git switch feature-login      # Switch
git switch -c feature-login   # Create + switch
```

💡 Recommended: learn the modern git switch syntax for branch operations.

23. git merge

Combines changes from one branch into another.

```
git switch main
git merge feature-login
```

⚠ **Direction matters.** If you're on main and run git merge feature-login, you're bringing feature-login **into** main.

24. git log

Shows commit history.

```
git log
```

Cleaner view:

```
git log --oneline
```

```
abc123 Add login page
def456 Add homepage
789xyz Initial commit
```

25. git diff

Shows differences between versions.

```
git diff
```

Think of it as: “What did I modify?”

26. git stash

Temporarily stores your uncommitted changes.

Scenario: You're working on feature-login with unfinished work, but there's a production bug to fix immediately.

```
git stash                # Save uncommitted work, clean working
directory
git switch main          # Switch and fix the bug
git switch feature-login
git stash pop            # Restore your changes
```

Uncommitted work → git stash → Temporarily stored → Do other work → git stash pop → Changes restored

27. git reset

Moves HEAD and can change what is staged or what is in your working directory, depending on the mode.

Unstage a file (keep file changes):

```
git reset HEAD file.txt
```

Three modes:

Mode	HEAD	Staged changes	Working changes
--soft	Moves	Remain staged	Kept
--mixed	Moves	Become unstaged	Kept
--hard	Moves	Discarded	Can be discarded

⚠ Be very careful with `git reset --hard` — it can **permanently discard uncommitted work**.

28. git revert

Creates a new commit that reverses an earlier commit.

Suppose commit C introduced a bug:

```
git revert C
```

A → B → C → D (D reverses the changes made by C)

This is generally **safer for shared branches** like main, since it doesn't rewrite history.

reset vs revert

(Important for DevOps interviews.)

Command	Effect
git reset	Moves branch history backward (A → B → C becomes A → B)
git revert	Creates a new commit that undoes previous changes (A → B → C becomes A → B → C → D)

For shared/public history, **revert is usually preferred** since it doesn't rewrite existing commit history.

GitHub

29. Create a GitHub Repository

Example repository structure:

```
devops-learning/  
├── Linux/  
├── Networking/  
├── AWS/  
├── Git/  
├── Docker/  
└── README.md
```

A good way to demonstrate your DevOps learning to recruiters.

30. SSH Authentication

SSH lets your computer authenticate with GitHub using a key pair instead of repeatedly entering credentials.

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

This creates: - **Private key** — stays on your computer - **Public key** — added to GitHub

| ⚠ Never share your private SSH key.

Repository URL over SSH:

```
git@github.com:username/project.git
```

Test the connection:

```
ssh -T git@github.com
```

31. Personal Access Token — PAT

A **Personal Access Token** is a credential used for GitHub authentication when a token is required instead of a password.

GitHub commonly uses **Username + Personal Access Token**, or preferably an **SSH key**, for Git operations.

| ⚠ Never publish your PAT in GitHub, a README, a Git repository, or screenshots. Treat it like a password.

32. Pull Request — PR

A **Pull Request** is a request to merge changes from one branch into another.

Developer flow:

Create feature branch → Write code → git add → git commit → git push
→ Create Pull Request

Another developer can then review code, comment, request changes, or approve.

feature-login → PR → main

33. Branch Protection

Branch protection rules help prevent dangerous changes to important branches (e.g., main), typically requiring:

Pull Request + Code Review + Status Checks → before merging

Typical professional workflow:

Developer → Feature branch → Pull Request → Code Review → CI tests → Approval → main

34. README

README.md explains what your project is about.

```
# AWS Learning

## Topics
- EC2
- S3
- IAM
- VPC
- EBS
- Auto Scaling

## Projects
- EC2 Web Server
- S3 Static Website
- VPC Architecture
```

GitHub automatically displays the README on your repository's main page — good READMEs show recruiters what you learned **and** what you actually practiced.

35. GitHub Actions Basics

This is where Git and DevOps start connecting directly.

GitHub Actions = CI/CD automation platform built into GitHub.

Developer → git push → GitHub → GitHub Actions → Test → Build → Deploy

A GitHub Actions workflow is stored at:

```
.github/
└─ workflows/
    └─ ci.yml
```

A workflow can say:

```
When code is pushed:
  ↓
Checkout code
  ↓
Install dependencies
  ↓
Run tests
  ↓
Build application
```

Later in your DevOps journey, this connects to:

Git → GitHub → GitHub Actions → Docker → AWS → Deployment

🔥 Complete Git Workflow Example

Building a website from scratch:

```
# Step 1 – Create project
mkdir my-website
cd my-website

# Step 2 – Initialize Git
git init

# Step 3 – Create files (index.html, style.css)
```

```

# Step 4 – Check status
git status

# Step 5 – Stage
git add .

# Step 6 – Commit
git commit -m "Initial website"

# Step 7 – Create a GitHub repository named "my-website"

# Step 8 – Connect local repo to GitHub (SSH)
git remote add origin git@github.com:USERNAME/my-website.git

# Step 9 – Push
git push -u origin main

```

Your Mac → Local Git Repository → git push → GitHub

🔥 Feature Development Example

Adding a login page:

```

# Create a branch
git switch -c feature-login

# Work on login.html, then check status
git status

# Stage and commit
git add login.html
git commit -m "Add login page"

# Push the branch
git push -u origin feature-login

```

Then on GitHub:

feature-login → Pull Request → Code Review → Approval → Merge → main

That's a real-world development workflow.

🔥 The Git Architecture You Should Memorize



|
WORKING DIRECTORY
Your actual files

For downloading:

GitHub
└─ git clone → First time download
└─ git fetch → Get remote information/changes without
integrating
└─ git pull → Fetch + integrate changes

Commands Cheat Sheet

Command	Purpose
git init	Create Git repository
git clone	Copy remote repository
git status	Check repository state
git add	Stage changes
git commit	Create checkpoint
git push	Upload commits
git pull	Fetch + integrate remote changes
git fetch	Download remote information/changes without integrating
git branch	View/create branches
git checkout	Older command for switching branches
git switch	Switch branches
git merge	Combine branches
git log	View commit history
git diff	View changes
git stash	Temporarily store uncommitted changes
git reset	Move/reset HEAD and potentially staging/worktree
git revert	Create a commit that reverses another commit

✧ Notes compiled for personal study — Git & GitHub fundamentals for DevOps.